

Mini-Cahier des Charges Définition du périmètre, fonctionnalités et architecture

Contexte et valeur ajoutée

1. CONTEXTE ET VALEUR AJOUTEE

SITUATION ACTUELLE

Actuellement, la location de voitures au Maroc se fait principalement de manière physique. Les clients doivent se rendre en agence, remplir des formulaires papier et attendre que leur demande soit traitée manuellement. Cela entraîne des coûts opérationnels élevés pour les agences et une expérience client médiocre.

PROBLEME RESOLU

CarRent propose une solution numérique pour l'ensemble du processus de location, de la recherche d'un véhicule à la confirmation de réservation, via une application mobile disponible sur Android et iO Les particuliers peuvent également mettre leur propre véhicule en location, créant ainsi un marché entre pairs.

VALEUR AJOUTEE CHIFFREE

Indicateur	Bénéfice estimé
Réduction du temps de réservation	De 45 min (agence) à < 3 min (app)
Taux de remplissage flotte agences	+25 % grâce à la visibilité numérique
Réduction des erreurs de gestion	-60 % (automatisation réservations)
Marché cible initial	Rabat-Salé-Kénitra (> 1,2 M habitants)

2. PARTIES PRENANTES

Partie prenante	Rôle	Besoins	Implication
Client final	Locataire de véhicule	Trouver/réserver rapidement un véhicule adapté	Forte – utilisateur principal
Agence de location	Loueur professionnel	Gérer flotte, réservations et revenus	Forte – fournisseur de contenu
Propriétaire particulier	Loueur pair-à-pair	Monétiser son véhicule facilement	Moyenne – profil secondaire
Administrateur	Gestionnaire plateforme	Modérer contenus et utilisateurs	Forte – contrôle qualité
Équipe de dev.	Développeurs / concepteurs	Livrer une app stable dans les délais	Forte – réalisation technique

3. PERIMETRE – MOSCOW

Priorité	Fonctionnalité	Description
Must	Authentification utilisateur	Inscription, connexion, déconnexion (email/password via Firebase)
Must	Recherche et filtrage de véhicules	Filtres par type, prix, disponibilité
Must	Réservation de véhicule	Sélection de dates, confirmation de réservation
Must	Mise en location d'un véhicule	Ajouter un véhicule avec photos, détails, tarif journalier
Must	Tableau de bord utilisateur	Historique des réservations, véhicules publiés
Should	Notifications push	Confirmation, rappel, annulation via Firebase FCM
Should	Système de notation et avis	Note (1-5 étoiles) après chaque location
Could	Paiement en ligne intégré	Intégration Stripe ou CMI Maroc
Could	Géolocalisation des véhicules	Carte interactive pour localiser les voitures proches
Won't	Module assurance intégré	Hors scope S6 – partenariat tiers requis

4. Contraintes

4.1 Contraintes techniques

- Framework : React Native (via Expo) – ce qui permet une utilisation multiplateforme sur Android et iOS.
- Backend : Firebase (Auth, Firestore, Storage) – sans serveur dédié
- Langage : JavaScript/TypeScript
- Version minimale : Android 8.0 (API 26), iOS 13
- Performance : temps de chargement page < 2 secondes sur 4G

4.2 Contraintes de délai

- Durée totale du projet : 12 séances (S1 à S12)
- Livraison finale (démonstration fonctionnelle) : Séance 12
- Équipe : 3 développeurs à temps partiel (~3h/semaine chacun)

4.3 Contraintes de sécurité

- Authentification sécurisée est assurée via Firebase Authentication qui utilise (hash bcrypt)
- Données personnelles qui conformes à la loi 09-08 qui concerne la protection des données, Maroc
- Aucune donnée bancaire stockée localement sur l'appareil
- Règles Firestore : accès en lecture/écriture restreint par rôle

5. Dépendances Externes

Service	Usage	Risque si indisponible	Alternative
Firestore Auth	Authentification	Aucun accès utilisateur	Expo Auth Sessions
Firestore DB	Stockage données	Perte accès véhicules/réservations	AsyncStorage local (offline)
Firestore Storage	Images véhicules	Images non affichées	Cloudinary (gratuit)
Expo (EAS Build)	Build & déploiement	Impossible de tester sur device	React Native CLI
Google Maps API	Géolocalisation (Could)	Fonctionnalité désactivée	Leaflet (open source)

6. Cas d'Utilisation

UC-01 : Réserver un véhicule

Champ	Détail
Acteur	Client final (utilisateur connecté)
Précondition	L'utilisateur est authentifié ; le véhicule est disponible aux dates souhaitées
Déroulement	1. Recherche un véhicule par type/ville 2. Consulte la fiche détaillée 3. Sélectionne dates début/fin 4. Confirme la réservation
Résultat	Réservation enregistrée dans Firestore ; notification de confirmation envoyée
Cas d'erreur	Véhicule déjà réservé → message d'erreur ; réseau indisponible → mode offline avec retry

UC-02 : Publier un véhicule à louer

Champ	Détail
Acteur	Propriétaire particulier ou agence
Précondition	L'utilisateur possède un compte vérifié
Déroulement	1. Accède au formulaire 'Ajouter un véhicule' 2. Renseigne marque, modèle, année, tarif 3. Sélectionne photo depuis la galerie 4. Soumet l'annonce
Résultat	Véhicule visible dans la liste principale ; image stockée sur Firestore Storage
Cas d'erreur	Champ obligatoire vide → validation inline ; image trop lourde (>5 Mo) → compression automatique

UC-03 : Gérer ses réservations

Champ Détail	
Acteur	Client final ou agence
Précondition	Au moins une réservation existante dans le compte
Déroulement	1. Accède à 'Mes Réservations' 2. Visualise liste triée par date 3. Sélectionne une réservation 4. Annule si statut = 'En attente'
Résultat	Statut mis à jour ; le véhicule redevient disponible
Cas d'erreur	Annulation hors délai (< 24h avant) → refus avec message explicatif

7. Architecture Technique

L'application repose sur une architecture 3 couches :

Couche	Technologie	Justification
Frontend (Mobile)	React Native + Expo	Développement multiplateforme rapide, large communauté, hot reload
Backend (BaaS)	Firebase (Auth + Firestore + FCM)	Pas de serveur à maintenir, scalabilité automatique, temps réel natif
Stockage Médias	Firebase Storage	CDN mondial, intégration native avec React Native, règles de sécurité Firestore
IDE / Outillage	VS Code + Expo CLI	Standard de l'industrie, débogage distant, preview QR code

Flux des données : Client (React Native) → Firebase SDK → Firestore/Storage → retour JSON temps réel au client.

8. Risques

Risque	Probabilité	Impact	Stratégie de mitigation
Indisponibilité Firebase	Faible	Élevé	Mode offline avec AsyncStorage + sync à la reconnexion
Sous-estimation des délais de dev.	Moyenne	Élevé	Réduction du périmètre (prioriser Must) sprints hebdomadaires
Mauvaise qualité des photos uploadées	Élevée	Moyen	Compression automatique côté client avant upload ; charte qualité
Faible de sécurité Firestore (règles mal configurées)	Moyenne	Élevé	Audit des règles Firestore à chaque sprint ; tests de pénétration basiques
Faible adoption utilisateurs	Moyenne	Moyen	Tests utilisateurs dès S6 ; amélioration UX basée sur retours

9. Glossaire

Terme	Définition dans le contexte CarRent
Locataire	Utilisateur qui effectue une réservation de véhicule via l'application

Loueur	Agence ou particulier qui met un véhicule à disposition sur la plateforme
Fiche véhicule	Écran détaillant marque, modèle, année, tarif, photos et disponibilité d'un véhicule
Réservation	Contrat numérique liant un locataire à un véhicule pour une plage de dates définie
Disponibilité	Statut d'un véhicule indiquant qu'il n'est pas réservé sur une période donnée
Firebase	Plateforme Google offrant des services BaaS : authentification, base de données NoSQL temps réel, stockage cloud
BaaS	Backend as a Service : infrastructure serveur gérée par un tiers (ici Firebase), sans déploiement serveur propre
MoSCoW	Méthode de priorisation : Must Have, Should Have, Could Have, Won't Have

10. Hypothèses et Exclusions

10.1 Hypothèses (ce qu'on suppose vrai)

- Chaque membre de l'équipe a accès à un poste de développement équipé de Node.js 18 ou supérieur, ainsi que d'Android Studio ou d'Expo Go installé.
- L'accès à Firebase est gratuit dans les limites du plan Spark, qui permet moins de 50 000 lectures par jour.
- Les utilisateurs que nous visons possèdent un smartphone Android ou iOS récent et ont accès à une connexion 4G.
- Les véhicules publiés sur notre plateforme sont vérifiés manuellement par les loueurs, ce qui signifie qu'il n'y a pas de contrôle automatique.
- Le projet est développé dans un contexte académique, ce qui signifie qu'il n'y aura pas de mise en production commerciale avant la fin du semestre 12.

10.2 Exclusions (ce que le projet NE fait PAS)

Voici les fonctionnalités qui ne sont pas incluses dans cette version :

- Il n'y a pas de module de paiement en ligne dans le périmètre S6, mais cela est prévu pour une version future.
- Il n'y a pas de vérification d'identité, ce que l'on appelle également connaissance du client, ou de scan de permis de conduire.
- Il n'y a pas d'interface web disponible, car l'application est uniquement conçue pour les appareils mobiles.
- Il n'y a pas de support pour plusieurs langues, donc l'application est seulement disponible en français et en anglais.
- Il n'y a pas de gestion de contrats juridiques ou de signatures électroniques.

11. KPIs – Indicateurs de Performance

#	KPI	Cible mesurable	Méthode de vérification (S12)
---	-----	-----------------	-------------------------------

1	Temps de réservation	< 3 minutes de bout en bout	Test chronométré sur 5 utilisateurs
2	Taux de succès des réservations	>= 95 % sans erreur technique	Scénario de test sur 20 réservations
3	Temps de chargement liste véhicules	< 2 secondes sur 4G	Mesure via Expo DevTools
4	Nombre de fonctionnalités Must livrées	5/5 Must Have opérationnels	Démo en direct à la soutenance S12
5	Note de satisfaction utilisateur	>= 4/5 étoiles (test utilisateurs)	Questionnaire SUS (5 testeurs)

12. Planification Macro (S1 – S12)

Phase	Séances	Activités principales	Livrables
Phase 1 – Initialisation	S1 – S2	Fiche projet, CDC, WBS, estimation, Gantt	Fiche projet, Mini-CDC, WBS, Gantt
Phase 2 – Conception	S3 – S4	Architecture Firebase, maquettes UI, diagrammes UML	Maquettes, diagramme de classes
Phase 3 – Développement	S5 – S8	Implémentation Auth, CRUD véhicules, réservations, tableau de bord	App fonctionnelle (Must Have)
Phase 4 – Tests	S9 – S10	Tests unitaires, tests d'intégration, corrections de bugs	Rapport de tests, app stabilisée
Phase 5 – Livraison	S11 – S12	Déploiement Expo EAS, documentation finale, préparation soutenance	APK/IPA, rapport final, démo

13. Critères de Succès

#	Critère (condition vérifiable)	Mode de vérification
1	Les 5 fonctionnalités Must Have sont démontrables sans crash en S12	Démo live devant le jury
2	Un utilisateur non initié peut effectuer une réservation complète en moins de 3 minutes	Test chronométré avec 3 volontaires
3	L'application fonctionne sur Android et iOS sans modification de code	Test sur 2 devices physiques (1 Android, 1 iOS)
4	Les données (véhicules, réservations) persistent entre 2 redémarrages de l'app	Test de persistance Firestore
5	Le code source est versionné sur GitHub avec au moins 20 commits documentés	Vérification du dépôt GitHub en soutenance

14. Wireframes – Écrans principaux

Les trois wireframes que vous trouverez ci-dessous montrent les fonctionnalités les plus importantes. Ils donnent une idée de la structure et de l'organisation des éléments, mais ne comprennent pas les détails de design comme les couleurs et les typographies, qui seront définis plus tard, lors de la phase 2.

WF-01 : Écran d'accueil – Recherche de véhicules

Structure de l'écran	Annotations
[Barre de navigation supérieure] Rechercher un véhicule... [Filtres : Tout SUV Sedan MPV Hatchback] Les plus loués : [Photo] Nom véhicule — Type/Transmission — \$XX /jour [Barre de navigation bas : Accueil Mes réservations Ajouter Profil]	• Barre de recherche → filtre en temps réel sur le catalogue Firestore • Chips de filtre → requête Firestore par champ 'type' • Liste scrollable → FlatList React Native, chargement paginé • Carte véhicule → tap → WF-03 (détail + réservation) • Navbar bas → navigation principale de l'app

WF-02 : Formulaire d'ajout d'un véhicule

Structure de l'écran	Annotations
< Retour — Ajouter un véhicule [Marque] [Modèle] [Année] [Type (SUV/Sedan/...)] [Tarif journalier (\$)] [Description] [Motorisation / Carburant] [Choisir une photo depuis la galerie] [AJOUTER LE VÉHICULE]	• Champs texte → validation inline obligatoire (non vide) • Picker type → dropdown liste fermée • Galerie → Expo ImagePicker, compression auto si > 2 Mo • Bouton ADD → upload Storage + écriture Firestore • Toast de confirmation → navigation vers WF-01

WF-03 : Détail véhicule et confirmation de réservation

Structure de l'écran	Annotations
< Retour — Détail [=== Photo du véhicule ===] Nom Modèle Année — \$XX /jour Description courte... Propriétés : Moteur: XXX hp Carburant: XX Cylindrée: XXXX cc Traction: XX [LOUER CE VÉHICULE] Date début : [Sélecteur date] Date fin : [Sélecteur date] [CONFIRMER LA RÉSERVATION]	• Photo → Image React Native avec fallback placeholder • Section propriétés → données Firestore formatées • Bouton LOUER → affiche les sélecteurs de date (DateTimePicker) • CONFIRMER → vérifie disponibilité + crée réservation dans Firestore • Succès → Dialog 'Réservation confirmée' + redirection Mes Réservations